

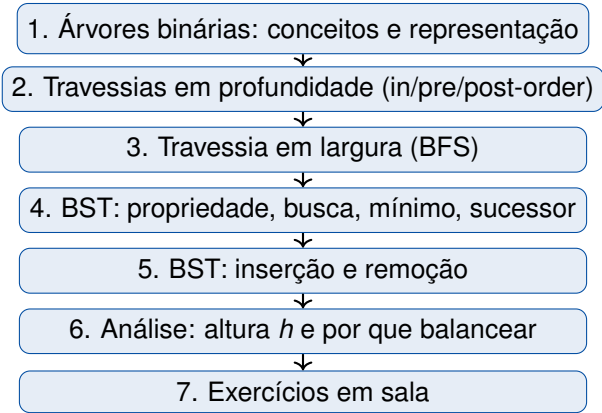
# Árvores Binárias de Busca (BST)

Aula 08 — COMP0497

Prof. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe  
Algoritmos e Estruturas de Dados 1 (COMP0497)

23 de maio de 2026



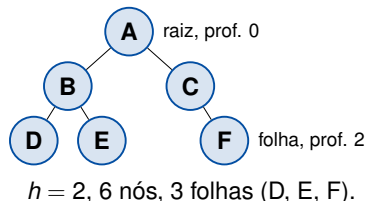
# Árvores binárias: conceitos e representação

## Definição

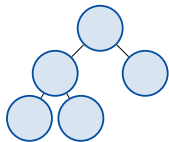
Uma **árvore binária** é um conjunto finito de nós que é *vazio*, ou consiste em um nó *raiz* mais duas árvores binárias disjuntas chamadas *subárvore esquerda* e *subárvore direita*.

## Terminologia:

- **Raiz:** único nó sem pai.
- **Filho / pai:** relação entre nós ligados.
- **Folha:** nó sem filhos.
- **Nó interno:** tem pelo menos um filho.
- **Irmãos:** filhos do mesmo pai.
- **Profundidade** de  $x$ : nº de arestas até a raiz.
- **Nível  $k$ :** todos os nós de profundidade  $k$ .
- **Altura  $h$ :** maior profundidade da árvore.

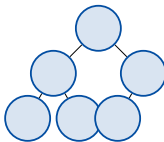


**Cheia (full)**



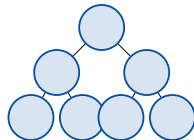
Todo nó tem 0 ou 2 filhos.

**Completa**



Todos os níveis cheios, exceto talvez o último (preenchido à esquerda).

**Perfeita**



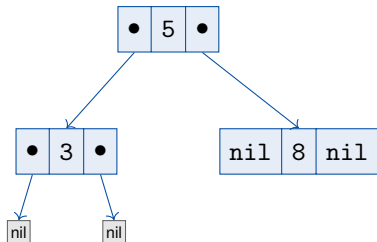
Todos os níveis completamente cheios.

## Limites de altura

Para uma árvore binária com  $n$  nós:

$$\lceil \log_2(n+1) \rceil - 1 \leq h \leq n - 1$$

O *mínimo* é atingido por árvores perfeitas; o *máximo*, por cadeias (uma sequência linear).



Cada célula: left | key | right

```
1  class Node<K, V> {
2      K key;
3      V value;
4      Node<K, V> left, right;    // filhos
5      Node<K, V> parent;        // opcional (CLRS)
6
7      Node(K key, V value) {
8          this.key = key;
9          this.value = value;
10     }
11 }
```

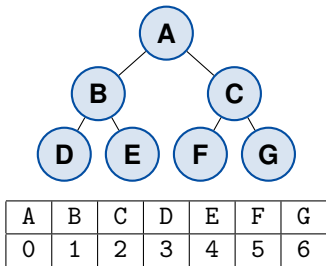
- **Sem** parent: estilo Sedgewick. Suficiente para BST.
- **Com** parent: estilo CLRS. Facilita sucessor e remoção iterativos.

Para árvores *completas* (heaps), pode-se guardar a árvore em um **vetor** sem ponteiros:

- raiz no índice 0;
- filhos de  $i$ :  $2i + 1$  (esq) e  $2i + 2$  (dir);
- pai de  $i$ :  $\lfloor (i - 1)/2 \rfloor$ .

## Quando usar?

Ótimo para **heaps** e árvores balanceadas estáticas. *Ruim* para BSTs gerais — desperdiça memória se a árvore for esparsa.



# Definição e propriedade da BST



## Problema:

Manter um *dicionário ordenado* com inserções, remoções e buscas frequentes.

- **Vetor ordenado:** busca  $O(\log n)$  **mas** inserção  $O(n)$ .
- **Lista encadeada:** inserção  $O(1)$  **mas** busca  $O(n)$ .

## Solução: BST

Estrutura ligada que mantém a ordem e oferece todas as operações em *tempo proporcional à altura  $h$* .

- Inserção, busca, remoção:  $O(h)$ .
- Em árvore balanceada:  
 $h = O(\log n)$ .

## Suporta

SEARCH, MIN, MAX, PREDECESSOR, SUCCESSOR, INSERT, DELETE

(Cormen et al., cap. 12)

## Propriedade da árvore binária de busca

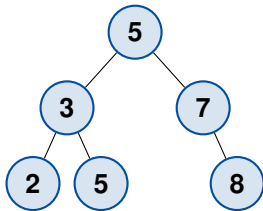
Para todo nó  $x$ :

- se  $y$  está na *subárvore esquerda* de  $x$ :  $key[y] \leq key[x]$ ;
- se  $y$  está na *subárvore direita* de  $x$ :  $key[y] \geq key[x]$ .

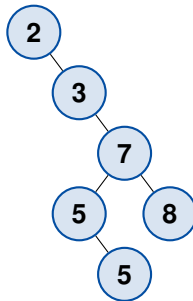
## Intuição

“Tudo à esquerda é menor (ou igual), tudo à direita é maior (ou igual).” Esta invariante vale *recursivamente* em cada subárvore.

**(a) altura 2 — eficiente**



**(b) altura 4 — ineficiente**



As mesmas chaves  $\{2,3,5,5,7,8\}$ , BSTs diferentes.  
O custo das operações depende de  $h$ , não de  $n$ .

Fig. 12.1, Cormen et al.

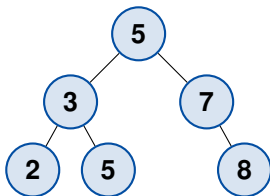
**Travessias**

## Problema:

Como visitar todos os nós em uma ordem útil?

**Solução:** três travessias recursivas naturais.

- **In-order:** *esq*, *raiz*, *dir*  $\Rightarrow$  ordem crescente.
- **Pre-order:** *raiz*, *esq*, *dir*.
- **Post-order:** *esq*, *dir*, *raiz*.

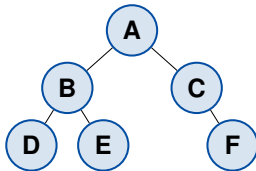


Saída: 2 3 5 5 7 8

```
1 void inorder(Node<K,V> x) {  
2     if (x == null) return;  
3     inorder(x.left);  
4     System.out.println(x.key);  
5     inorder(x.right);  
6 }
```

## Teorema 12.1 (Cormen)

INORDER-TREE-WALK executa em  $\Theta(n)$  em uma BST com  $n$  nós.



**pre-order:** A B D E C F  
**post-order:** D E B F C A

```
1 void preorder(Node<K,V> x) {  
2     if (x == null) return;  
3     visit(x);                      // raiz primeiro  
4     preorder(x.left);  
5     preorder(x.right);  
6 }  
7  
8 void postorder(Node<K,V> x) {  
9     if (x == null) return;  
10    postorder(x.left);  
11    postorder(x.right);  
12    visit(x);                      // raiz por ultimo  
13 }
```

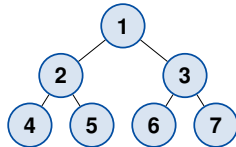
- **Pre-order:** cópia de árvores, serialização.
- **Post-order:** liberar memória, avaliar expressão (folhas antes da raiz).

# Travessia em largura (BFS)



**Ideia:** usar uma *fila* (FIFO). Visite a raiz, enfileire os filhos, repita.

- Visita os nós por **nível crescente** de profundidade.
- Custo:  $\Theta(n)$  tempo,  $O(\text{largura máxima})$  espaço.
- Em árvore perfeita: largura  $\approx n/2$ .



Ordem BFS: 1 2 3 4 5 6 7  
(em contraste: pre-order = 1 2 4 5 3 6 7)

## Quando usar BFS?

- Encontrar nó mais próximo da raiz com certa propriedade.
- Calcular nível de cada nó.
- Imprimir árvore "nível a nível".

```
1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public void bfs(Node<K, V> root) {
5     if (root == null) return;
6     Deque<Node<K, V>> fila = new ArrayDeque<>();
7     fila.offer(root);           // enfileira raiz
8     while (!fila.isEmpty()) {
9         Node<K, V> x = fila.poll(); // remove da frente
10        System.out.println(x.key);
11        if (x.left != null) fila.offer(x.left);
12        if (x.right != null) fila.offer(x.right);
13    }
14 }
```

## DFS vs BFS

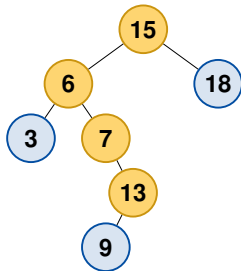
**DFS** (in/pre/post-order) usa *pilha* (recursão); explora um ramo de cada vez.

**BFS** usa *fila*; explora a árvore em camadas.

**Busca, mínimo, máximo,  
sucesor**

## Ideia

Compare  $k$  com a chave de cada nó. Se  $k < \text{key}[x]$ , vá para a esquerda; se  $k > \text{key}[x]$ , vá para a direita.



Buscando 13:  $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$

## Custo

$O(h)$  comparações. Em BST balanceada:  $O(\log n)$ .

## Versão iterativa (preferida em Java)

Evita estouro de pilha em árvores muito altas e é geralmente mais rápida.

```
1 public class BST<Key extends Comparable<Key>, Value> {
2
3     public Value get(Key key) {
4         Node<Key,Value> x = root;
5         while (x != null) {
6             int cmp = key.compareTo(x.key);
7             if (cmp < 0) x = x.left;
8             else if (cmp > 0) x = x.right;
9             else return x.value; // achou
10        }
11        return null; // ausente
12    }
13 }
```

- `Key extends Comparable<Key>`: aceita qualquer tipo ordenável.
- Versão iterativa,  $O(h)$  tempo,  $O(1)$  espaço extra.

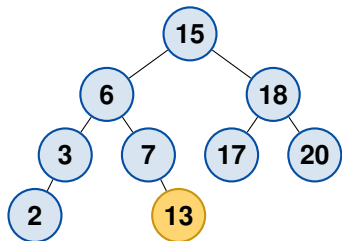
**Mínimo:** desça pela esquerda até null.

**Máximo:** desça pela direita até null.

**Sucessor de  $x$  (próxima chave em ordem):**

- se  $x$  tem subárvore direita: *mínimo* dessa subárvore;
- caso contrário: *primeiro ancestral* cujo filho esquerdo também é ancestral de  $x$ .

Tempo:  $O(h)$  em ambos os casos.



Sucessor de 13 é 15 (sobe pelos ancestrais).

# Operações ordenadas: rank e select

## Por que manter `size` em cada nó?

`x.size = 1 + size(x.left) + size(x.right)` permite consultas *ordinais* em  $O(h)$ :

- **rank(key)**: quantas chaves são *menores* que `key`?
- **select(k)**: qual é a  $k$ -ésima menor chave? (índice 0)

```
1 public int rank(Key key) {
2     return rank(root, key);
3 }
4 private int rank(Node<Key,Value> x, Key key) {
5     if (x == null) return 0;
6     int cmp = key.compareTo(x.key);
7     if (cmp < 0) return rank(x.left, key);
8     else if (cmp > 0) return 1 + size(x.left)
9                     + rank(x.right, key);
10    else return size(x.left);
11 }
```

```
1 public Key select(int k) {
2     return select(root, k).key;
3 }
4 private Node<Key,Value> select(Node<Key,Value> x
5     , int k) {
6     if (x == null) return null;
7     int t = size(x.left);
8     if (t > k) return select(x.left, k);
9     else if (t < k) return select(x.right, k - t
10        - 1);
11    else return x;
12 }
```

Sedgewick & Wayne, *Algorithms* §3.2 — também útil para *range queries* em  $O(\log n + k)$ .



# **Inserção e remoção**

## Ideia

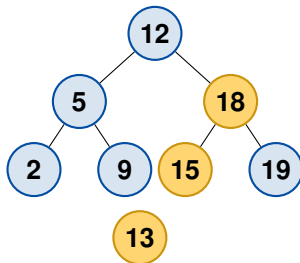
Buscar pela chave; ao chegar em null, criar nó. A versão recursiva *retorna a subárvore* e atualiza ponteiros ao subir.

```
1 public void put(Key key, Value value) {
2     root = put(root, key, value);
3 }
4 private Node<Key,Value> put(Node<Key,Value> x, Key key, Value value) {
5     if (x == null) return new Node<>(key, value);
6     int cmp = key.compareTo(x.key);
7     if (cmp < 0) x.left = put(x.left, key, value);
8     else if (cmp > 0) x.right = put(x.right, key, value);
9     else x.value = value; // chave ja existe
10    x.size = 1 + size(x.left) + size(x.right); // mantem tamanho
11    return x;
12 }
```

Custo:  $O(h)$ .

(Sedgewick, cap. 12)

Inserindo **13** em uma BST existente.

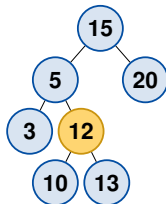


O caminho  $12 \rightarrow 18 \rightarrow 15$  leva à posição correta. Aresta tracejada é a nova ligação.

Fig. 12.3, Cormen et al.

Seja  $z$  o nó a remover.

1.  $z$  **sem filhos**: basta desligar  $z$  do pai.
2.  $z$  **com 1 filho**: pai de  $z$  aponta para o único filho de  $z$ .
3.  $z$  **com 2 filhos**: substitua  $z$  pelo seu *sucessor in-order*  $y$  (o mínimo da subárvore direita);  $y$  tem no máximo 1 filho.



Remover **12**: sucessor é 13; copiar 13 para o lugar de 12 e remover o nó 13 original.

Fig. 12.4, Cormen et al. — método de Hibbard

```
1 private Node<Key,Value> delete(Node<Key,Value> x, Key key) {
2     if (x == null) return null;
3     int cmp = key.compareTo(x.key);
4     if (cmp < 0) x.left = delete(x.left, key);
5     else if (cmp > 0) x.right = delete(x.right, key);
6     else {
7         if (x.right == null) return x.left;           // caso 1 ou 2
8         if (x.left == null) return x.right;           // caso 2
9         Node<Key,Value> t = x;                         // caso 3
10        x = min(t.right);                             // sucessor
11        x.right = deleteMin(t.right);
12        x.left = t.left;
13    }
14    x.size = 1 + size(x.left) + size(x.right);
15    return x;
16 }
```

**Análise: altura e por que  
balancear**

## Teorema (Cormen 12.4 / Aslam)

A altura esperada de uma BST construída *aleatoriamente* por  $n$  inserções de chaves distintas é  $\mathbb{E}[h] = O(\log n)$ .

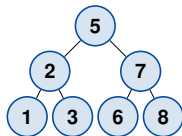
## Mas no pior caso

Inserindo chaves em ordem crescente, a BST vira uma lista encadeada:  $h = n - 1$  e todas as operações custam  $\Theta(n)$ .

| Operação                 | BST balanceada | BST degenerada |
|--------------------------|----------------|----------------|
| SEARCH / INSERT / DELETE | $O(\log n)$    | $\Theta(n)$    |
| MIN / MAX / SUCCESSOR    | $O(\log n)$    | $\Theta(n)$    |

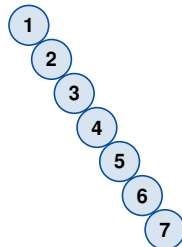
# Exemplo: efeito da ordem de inserção

**Aleatória** {5,2,7,1,3,6,8}



$h = 2$ , balanceada

**Ordenada** {1,2,3,4,5,6,7}



$h = 6$ , degenerada

## Motivação para a próxima aula

Árvores balanceadas (AVL, rubro-negra) garantem  $h = O(\log n)$  **no pior caso**.



# Demonstração: a árvore vira uma lista

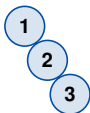
Inserindo 1, 2, 3, 4, 5 *em ordem*, a cada passo:



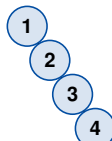
após 1



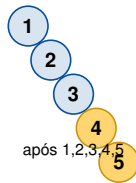
após 1,2



após 1,2,3



após 1,2,3,4



após 1,2,3,4,5

## Observe

A árvore degenera em uma **lista encadeada**:  $h = n - 1$ , busca  $\Theta(n)$ . O mesmo conjunto de chaves em ordem aleatória dá  $h \approx 2$ . A *ordem de inserção* é tudo.

## O Java já implementa BST balanceada

`TreeMap<K,V>` e `TreeSet<E>` são *árvores rubro-negras*: garantem  $O(\log n)$  no pior caso e ordenação automática das chaves.

```
1 import java.util.TreeMap;
2 import java.util.NavigableMap;
3
4 NavigableMap<String, Integer> agenda = new TreeMap<>();
5 agenda.put("Ana", 11);
6 agenda.put("Bruno", 55);
7 agenda.put("Carla", 99);
8
9 agenda.firstKey();           // "Ana"           -- min
10 agenda.lastKey();           // "Carla"          -- max
11 agenda.higherKey("Bruno");  // "Carla"          -- sucessor estrito
12 agenda.headMap("Carla");    // {Ana=11, Bruno=55} -- range query
```

## Quando reimplementar?

*Quase nunca em produção* — use `TreeMap`. Implementamos BST do zero para **entender** a estrutura e poder modificá-la (ex.: AVL, treap, BST aumentada para intervalos).

# Exercícios em sala

## Enunciado

Insira na BST inicialmente vazia, *nesta ordem*, as chaves:

50, 30, 70, 20, 40, 60, 80, 35, 45

## Pedidos:

1. Desenhe a BST resultante.
2. Liste as chaves em *in-order*, *pre-order* e *post-order*.
3. Liste a ordem da travessia *BFS* (nível a nível).
4. Qual é a altura  $h$  da árvore?

## Dica

A travessia *in-order* deve produzir as chaves em ordem crescente — use isso para conferir seu desenho.

### Use a árvore do Exercício 1

1. Quantas comparações são feitas ao buscar a chave 45? E a chave 55?
2. Qual é o sucessor in-order de 40? E o sucessor de 50?
3. Remova a chave 30 (nó com dois filhos) usando o método de Hibbard. Desenhe a árvore resultante.
4. Remova a chave 70. Desenhe a árvore resultante.
5. A altura mudou? Como?

### Parte A — pior caso

Insira em uma BST vazia, *nesta ordem*: 1, 2, 3, 4, 5, 6, 7.

1. Desenhe a árvore.
2. Quantas comparações são necessárias para buscar a chave 7?
3. Quanto custaria a mesma busca em uma BST balanceada com as mesmas chaves?

### Parte B — código

Escreva, em Java, um método `int contarFolhas(Node<K,V> x)` que retorna o número de folhas de uma subárvore.

*Resposta esperada:* 4 funções de uma linha + caso-base. Discuta a complexidade.

# Recapitulação

## O que aprendemos hoje

- **Árvore binária:** terminologia (raiz, folha, altura), tipos (cheia, completa, perfeita) e duas representações (ligada e por array).
- **Travessias DFS:** in-order (ordenado), pre-order, post-order; todas em  $\Theta(n)$ .
- **Travessia BFS:** nível a nível, usando fila (ArrayDeque).
- **Propriedade da BST:** esquerda  $\leq$  raiz  $\leq$  direita.
- **Search, Min, Max, Successor, Insert, Delete** custam  $O(h)$ .
- **Inserção (Sedgwick)** recursiva retornando a subárvore; **Remoção (Hibbard)** usa sucessor in-order.
- **Altura** é tudo:  $O(\log n)$  esperado,  $\Theta(n)$  no pior caso  $\Rightarrow$  motiva árvores balanceadas.

## Próxima aula

Árvores balanceadas (AVL): rotações que mantêm  $h = O(\log n)$  no pior caso.



